Análise das estruturas de dados e dos algoritmos implementados em Java e Python

O projeto reúne 24 situações-problema, com 12 implementações em Java e 12 em Python. A numeração adotada neste relatório é a das situações-problema do projeto. No conjunto Java, ela difere da numeração usada pelo menu da classe `Main`, que encaminha a execução às demonstrações e aos testes de cada implementação. Os exemplos a seguir identificam os dados fornecidos às operações ou às demonstrações e apresentam os resultados correspondentes.

Na análise de complexidade, *n* representa a quantidade de elementos da estrutura em questão, salvo indicação específica. Os custos de armazenamento da estrutura, da memória de trabalho e do resultado são distinguidos sempre que essa separação afeta a interpretação. Nas expressões que contam comparações, chaves de tamanho limitado são tratadas como operações de custo constante. Quando o comprimento de textos, a distribuição das chaves ou a disposição inicial dos dados influencia o resultado, a hipótese é explicitada. Os desafios discutidos decorrem das invariantes e dos casos especiais observáveis no código-fonte.

1. Implementações em Java

1.1. Situação 01 - Atendimento de pacientes em uma clínica

A classe `Clinica` simula o atendimento de pacientes na ordem em que foram inseridos. A classe genérica `Fila<T>` implementa a disciplina FIFO por meio de nós simplesmente encadeados e mantém referências para `inicio` e `fim`, além de um contador de elementos. `enqueue` liga o novo nó diretamente ao último, enquanto `dequeue` avança o início. Ao retirar o único elemento, a operação também limpa a referência para o fim. Essa atualização preserva a consistência da fila quando ela volta a receber pacientes. `front` consulta o primeiro registro sem removê-lo, e a classe sinaliza por exceção as tentativas de consultar ou retirar de uma fila vazia.

Cada operação da fila custa *O(1)* em tempo e usa *O(1)* de memória adicional por chamada. O armazenamento de *n* pacientes enfileirados requer *O(n)* nós. A simulação percorre o vetor de pacientes uma vez para enfileirá-los e depois esvazia a fila, com custo total *O(n)* em qualquer ordem de entrada. O relatório acumula a espera em uma variável e calcula a média ao final, de modo que os cálculos exigem *O(1)* de memória além do vetor e da fila. A espera depende do horário de chegada e da disponibilidade após o atendimento anterior. O tipo de atendimento é exibido, e todos os atendimentos têm duração estimada de 15 minutos.

Na demonstração com 15 pacientes, Ana chega às 08h05 e é atendida no mesmo horário. Bruno chega às 08h12 e aguarda até 08h20, o que produz uma espera estimada de oito minutos. Carla, que chega às 08h15, é atendida às 08h35 e espera 20 minutos. Para o conjunto completo, o programa informa `Tempo medio estimado de espera: 59.07 min` e `Tamanho final da fila: 0`. O cálculo do início de cada atendimento também contempla períodos ociosos, pois usa o horário de chegada quando ele é posterior à disponibilidade da clínica.

1.2. Situação 03 - Histórico de alterações de um editor de texto

`EditorTexto` implementa digitação ao final do documento, exclusão por intervalo e mudança entre os formatos disponíveis em `FormatoTexto`. Cada alteração produz um `EstadoEditor` com texto e formato, e `registrar` insere uma `Acao` em `PilhaAcoes`. A ação referencia os estados anterior e posterior. A pilha encadeada segue a disciplina LIFO, adequada porque o estado a recuperar pertence à alteração mais recente. `desfazer` remove a ação do topo e atribui o estado anterior ao editor. A exclusão valida o intervalo semiaberto *[inicio,fim)*, e as conversões para maiúsculas ou minúsculas usam `Locale.ROOT`. A opção `NORMAL` preserva a sequência atual de caracteres e altera o indicador de formato.

`push`, `pop` e `desfazer` custam *O(1)* em tempo e espaço de trabalho. O custo de editar o texto é diferente do custo de registrar ou desfazer a ação. Se o documento tem comprimento *L* e são digitados *d* caracteres, a concatenação custa *O(L+d)* em tempo e memória para o novo texto e a descrição. A exclusão e as conversões de caixa têm custo de até *O(L)*. Com *a* ações armazenadas, o histórico ocupa *O(a)* registros, além da memória dos textos e descrições retidos. Uma sequência que acrescenta um caractere por ação pode conservar versões de comprimentos crescentes e consumir $O(a^2)$ caracteres. O armazenamento dos estados completos torna a restauração direta. Um histórico baseado em operações inversas poderia reduzir a duplicação de texto, com a necessidade de reconstruir cada estado ao desfazer.

Na demonstração de `Problema2`, digitar `Ola`, `, mundo` e `!` produz `Ola, mundo!`. A operação seguinte muda o estado para `texto="OLA, MUNDO!", formato=MAIUSCULAS`. Ao desfazer essa formatação, depois de já terem sido desfeitas as alterações posteriores, o programa exibe `Desfeito -> texto="Ola, mundo!", formato=NORMAL`. Ao esgotar as 12 ações, o texto volta a `""`, com formato `NORMAL`, e uma nova chamada a `desfazer` retorna `false`.

1.3. Situação 05 - Gerenciamento de uma lista de reprodução

`Playlist` organiza objetos `Musica` em uma lista simplesmente encadeada. `inserirInicio` e `inserirFinal` usam as referências das extremidades, enquanto `inserir` admite posições a partir de zero e percorre os nós até o predecessor quando a posição é interna. `removerPorTitulo` exclui a primeira ocorrência que coincide com o título, ignorando diferenças de caixa. A operação mantém as referências para o nó atual e para o anterior, o que permite religar a cadeia e ajustar o fim quando necessário. `buscar` procura um trecho no título ou no artista. A primeira passagem conta as correspondências, e a segunda preenche um vetor do tamanho exato do resultado.

As inserções nas extremidades custam *O(1)* em tempo. A inserção interna na posição *p* custa *O(p)*, e a remoção por título varia de *O(1)*, quando o primeiro nó corresponde à consulta, a *O(n)*. `listar` percorre *n* nós e produz um vetor de *n* referências. `buscar` executa duas varreduras completas, com *O(n)* visitas e *O(k)* posições de saída para *k* correspondências, além do trabalho de converter e pesquisar as cadeias de caracteres. A lista ocupa *O(n)* nós. Um vetor permitiria acesso direto por posição, mas deslocaria elementos em inserções ou remoções internas.

`reordenarPorTitulo` implementa ordenação por inserção com religação de nós. A cada iteração, guarda o próximo nó da sequência original e encaixa o atual em uma cadeia ordenada. O novo nó fica depois dos títulos equivalentes, preservando a ordem relativa em empates. A ordenação usa *O(1)* de memória adicional e custa *O(n)* no melhor caso, como na entrada em ordem estritamente decrescente, em que cada nó entra no início da cadeia ordenada. A entrada crescente exige percorrer o prefixo a cada inserção, produzindo $O(n^2)$. Uma permutação aleatória de títulos distintos também requer $O(n^2)$ em média.

Em `PlaylistApp`, `Tempo Perdido` já está no conjunto inicial e é inserida novamente na posição 4. Depois da remoção de `Pais e Filhos`, a consulta `Legiao` devolve duas ocorrências de `Tempo Perdido`. O resultado

decorre da busca também pelo artista e da possibilidade de armazenar músicas repetidas. Após a reordenação, as duas ocorrências ocupam posições consecutivas.

1.4. Situação 07 - Cadastro de alunos por matrícula

`ArvoreAlunos` indexa objetos `Aluno` pela matrícula inteira. Em cada nó da árvore binária de busca, as matrículas menores ficam à esquerda e as maiores à direita. `inserir` e `buscar` comparam a chave com a do nó corrente e prosseguem por um único ramo. Uma tentativa de cadastrar matrícula já existente retorna `false`. A listagem em ordem visita a subárvore esquerda, o nó e a subárvore direita, produzindo diretamente a sequência crescente de matrículas.

O desafio da remoção consiste em preservar a ordenação da árvore quando o registro tem dois filhos. Nesse caso, `remover` copia para o nó o aluno com a menor matrícula da subárvore direita e chama `removerSucessor` para retirar a ocorrência original. O contador é decrementado na retirada efetiva. Em uma remoção com até um filho, a função devolve diretamente a referência que deve ocupar o lugar do nó retirado.

Para uma árvore de altura *h*, inserção, busca e remoção custam até *O(h)*. O melhor caso é *O(1)* quando a operação se resolve junto à raiz, como em uma busca pela própria raiz. Sob uma ordem aleatória de inserção de chaves distintas, o custo médio de percorrer um caminho é *O(log n)*. Uma sequência crescente ou decrescente pode produzir altura *n* e elevar uma operação a *O(n)* e a construção da árvore a $O(n^2)$. Inserção e busca usam *O(1)* de espaço auxiliar, enquanto a remoção recursiva e a listagem podem usar *O(h)* na pilha. A listagem visita todos os nós em *O(n)*, e o índice ocupa *O(n)* nós.

Na demonstração com 20 alunos, buscar `202315` encontra Carla Mendes, do curso `Ciencia da Computacao`, e buscar `202399` resulta em `Matricula 202399 nao encontrada.`. A remoção de `202305` é confirmada e a matrícula deixa de aparecer no percurso ordenado. O teste de remoção com dois filhos insere as chaves 30, 20, 40 e 10. Após excluir 30, a busca por essa chave devolve `null` e o tamanho é três.

1.5. Situação 09 - Índice de funcionários por CPF com árvore AVL

`ArvoreAVL` organiza registros de `Funcionario` por CPF e armazena a altura de cada nó. A representação do CPF como cadeia de exatamente 11 dígitos preserva zeros iniciais e permite comparar as chaves diretamente. A validação verifica a forma da chave. Inserções duplicadas geram exceção. A cada inserção ou remoção, o retorno da recursão atualiza as alturas e calcula o fator de balanceamento pela diferença entre as alturas esquerda e direita. Se o fator sai do intervalo de *-1* a *1*, `balancear` aplica uma rotação simples ou dupla, conforme a inclinação da subárvore. As rotações atualizam primeiro a altura do nó que desceu e depois a da nova raiz local.

A árvore ocupa *O(n)* nós. Como o balanceamento limita a altura a *O(log n)*, buscas, inserções e remoções têm esse limite de tempo no pior caso. Uma busca pela raiz custa *O(1)*. Nas alterações bem-sucedidas, o código percorre o caminho até a inserção ou retirada física e faz trabalho constante por nível. A remoção primeiro procura o CPF e, se ele existir, percorre novamente a árvore para retirá-lo. As duas passagens continuam limitadas por *O(log n)*, com até *O(log n)* de pilha auxiliar. Uma rotação isolada custa *O(1)*. `altura` consulta um valor já armazenado, enquanto `estaBalanceada` percorre a árvore válida em *O(n)* para verificar fatores e alturas. `listarEmOrdem` também custa *O(n)* e devolve um vetor de *n* referências.

Em `SistemaRH`, a inserção de 25 CPFs consecutivos produz altura 5, fator 0 na raiz e confirmação de que a árvore está balanceada. A consulta `10000000013` devolve o funcionário 13, cujo cargo é `Assistente`. A remoção de `10000000010` é confirmada, seguida de `Balanceada apos a operacao? true`. O conjunto crescente de entrada evidencia a função das rotações, pois a mesma sequência de chaves formaria uma cadeia em uma árvore binária de busca sem rebalanceamento.

1.6. Situação 11 - Roteamento de entregas com Dijkstra

`GrafoPonderado` representa a rede de entregas por listas de adjacência. Os nomes dos vértices ocupam um vetor de capacidade fixa e cada conexão acrescenta um nó de aresta em cada sentido. A estrutura rejeita pesos negativos ou não finitos, o que preserva a hipótese de pesos não negativos exigida pela estratégia de Dijkstra. `Dijkstra.encontrar` mantém vetores de distâncias, predecessores e marcações de visita. A cada passo, seleciona por varredura o vértice não visitado de menor distância e relaxa suas arestas. Se encontra uma distância estritamente menor para um vizinho, atualiza o custo e o predecessor. Ao final, reconstrói a rota percorrendo os predecessores do destino até a origem e invertendo essa sequência.

Para *V* vértices e *E* conexões, as listas de adjacência ocupam *O(C+E)*, em que *C* é a capacidade reservada para vértices. A consulta usa *O(V)* de espaço auxiliar e tempo $O(V^2+E)$. Na implementação, `selecionarMenorNaoVisitado` também pode escolher um vértice cuja distância é infinita. Por isso, em uma consulta válida, a seleção examina o vetor inteiro nos *V* passos, inclusive em grafos desconexos ou quando o destino já foi visitado. A busca inicial dos nomes é linear em *V*, e a reconstrução acrescenta até *O(V)*. A montagem do grafo também faz buscas lineares de nomes para detectar vértices repetidos e localizar os extremos de cada conexão. Uma fila de prioridade poderia reduzir o custo da seleção em grafos esparsos, ao passo que a varredura adotada requer apenas vetores de tamanho *V* e operações diretas por índice.

Na demonstração, a consulta de `Centro de Distribuicao` até `Aeroporto` devolve `Centro de Distribuicao -> Sul -> Industrial -> Porto -> Aeroporto`, com custo `11.0`. O total corresponde à soma dos pesos 2, 3, 4 e 2. Quando o destino permanece com distância infinita, `encontrar` devolve um objeto `Rota` que indica ausência de caminho.

1.7. Situação 14 - Contagem de frequência de palavras

`ContadorPalavras.processar` converte a entrada para minúsculas e aplica uma expressão regular que reconhece letras, marcas de combinação e dígitos, permitindo hífens internos entre grupos de caracteres. Cada ocorrência é entregue a `TabelaHash.adicionar`. A tabela calcula um hash polinomial com multiplicador 31 e resolve colisões com encadeamento separado. Ao localizar uma palavra existente no balde, incrementa sua frequência. Uma palavra nova recebe um nó. A distinção entre o total de ocorrências e a quantidade de palavras distintas permite registrar as duas medidas sem repetir a varredura do texto. A capacidade padrão de 101 baldes é fixa.

Se o texto tem comprimento *L*, contém *t* ocorrências e *u* palavras distintas, a normalização, a identificação das ocorrências e o cálculo dos hashes envolvem trabalho linear no volume de caracteres, ao qual se soma a procura nas cadeias. Sob distribuição aproximadamente uniforme e para comparações de chaves de tamanho limitado, um limite esperado é *O(L+t(1+u/C))*, em que *C* é a capacidade da tabela. Com cadeias curtas, a contagem se aproxima de *O(L+t)*. No pior caso, muitas palavras podem cair em um único balde, levando a *O(L+tu)* no mesmo modelo. O índice ocupa *O(C+u)* referências e nós, além dos textos das palavras distintas. A versão normalizada da entrada pode exigir *O(L)* de memória de trabalho.

`maisFrequentes` percorre os baldes, cria um vetor dos pares e usa ordenação por inserção em ordem decrescente de frequência, com desempate lexicográfico. A operação custa *O(C+u)* no melhor caso, quando o vetor já segue essa ordem, e $O(C+u^2)$ no pior caso. Como ordena todos os pares antes do recorte, pedir poucos resultados reduz o tamanho da saída, mas conserva o custo da ordenação. A chamada usa *O(u)* de espaço adicional para materializar os pares e a saída.

Ao processar o preâmbulo reproduzido em `Problema7`, o programa conta 80 ocorrências e 64 palavras distintas. Entre os resultados estão `a: 8`, `e: 5` e `o: 3`. O item `bem-estar: 1` confirma que o hífen interno é conservado na palavra.

1.8. Situação 15 - Ordenação de notas e comparação entre algoritmos

`Problema8` usa três cópias do mesmo vetor de `NotaAluno` para comparar `BubbleSortNotas`, `SelectionSortNotas` e `InsertionSortNotas`. Os três aplicam o mesmo critério, com notas crescentes e desempate pelo nome do aluno sem distinção de caixa. `EstatisticasOrdenacao` contabiliza comparações e trocas. Ao manter os dados iniciais e o comparador iguais, a demonstração permite atribuir as diferenças das contagens às estratégias de movimentação e busca de posição.

O Bubble Sort troca vizinhos fora de ordem, reduz a região percorrida após cada passagem e encerra ao completar uma passagem sem trocas. Seu melhor caso é *O(n)*, com *n-1* comparações para um vetor já ordenado com pelo menos dois elementos. O caso médio para uma permutação aleatória de chaves distintas e o pior caso são $O(n^2)$. O Selection Sort examina todo o sufixo para escolher o menor elemento, de modo que faz *n(n-1)/2* comparações em qualquer ordem de entrada e no máximo *n-1* trocas. O Insertion Sort desloca o elemento corrente para a esquerda por trocas adjacentes. Se há *i* inversões segundo o comparador, faz *i* trocas e tem custo *O(n+i)*, linear no melhor caso e quadrático no caso médio aleatório e no pior caso. Bubble e Insertion preservam a ordem relativa de elementos equivalentes pelo comparador. Selection pode alterá-la em uma troca distante.

Cada algoritmo modifica o vetor recebido e usa *O(1)* de memória de trabalho. As cópias produzidas pela demonstração custam *O(n)* de espaço adicional. A validação percorre o vetor e rejeita entradas com mais de 30 alunos, portanto as ordens de crescimento descrevem os laços dentro do tamanho admitido pela implementação.

A entrada da demonstração contém, na ordem, Ana com 7,50, Bruno com 5,00, Carla com 9,00, Diego com 6,50, Elisa com 8,00, Fabio com 5,00, Gabriela com 10,00 e Henrique com 4,50. O vetor ordenado começa por Henrique, Bruno e Fabio e termina com Carla e Gabriela. Para essa mesma entrada, Bubble Sort registra 28 comparações e 15 trocas, Selection Sort registra 28 comparações e cinco trocas, e Insertion Sort registra 20 comparações e 15 trocas.

1.9. Situação 17 - Classificação de candidatos e extração dos primeiros colocados

`SelectionSort.ordenar` organiza um vetor de candidatos por pontuação decrescente e, no empate, pelo nome em ordem crescente, sem distinção de caixa. Em cada passagem, procura o melhor candidato do sufixo e o coloca na posição corrente por uma única troca, quando necessária. Essa estratégia limita a quantidade de trocas, embora examine todo o conjunto restante a cada passagem. `extrairTopN` valida os elementos e o tamanho solicitado, verifica a ordenação de todo o vetor e copia o prefixo para um vetor de saída.

A ordenação usa *O(1)* de espaço auxiliar e custa $O(n^2)$ nos casos melhor, médio e pior, pois sempre executa *n(n-1)/2* comparações. Para um prefixo de tamanho *k*, a extração custa *O(n+k)* em uma entrada válida, e o vetor de saída usa *O(k)*. O custo total continua quadrático porque a demonstração ordena a classificação completa antes de extrair os primeiros colocados. Uma fila de prioridade limitada a *k* elementos seria uma alternativa para uma consulta restrita aos melhores candidatos. A ordenação completa é pertinente ao programa, que imprime também a classificação geral.

Na demonstração sem argumento, `Problema9` utiliza 15 candidatos e solicita os cinco primeiros. O resultado é Eduarda Rocha, Mariana Santos, Bruno Lima, Gabriela Martins e Patricia Gomes, nessa ordem. Eduarda e Mariana têm 95,5 pontos, enquanto Bruno e Gabriela têm 91,0. Nos dois pares, o comparador decide a posição pelo nome. Patricia completa a seleção com 90,0 pontos.

1.10. Situação 19 - Inserção ordenada de cartas

`MaoCartas` mantém um vetor de referências a objetos `Carta` e preserva a mão em ordem crescente após cada recebimento. `inserir` começa pela última posição ocupada e desloca para a direita as cartas de valor estritamente maior. A nova carta ocupa a posição liberada, depois de eventuais cartas do mesmo valor. A operação corresponde à etapa de inserção do Insertion Sort, aplicada a um prefixo que já está ordenado. O acesso por índice em `get` é *O(1)*, e `garantirCapacidade` duplica o vetor quando a capacidade, inicialmente 13, é atingida.

Com *n* cartas na mão, uma inserção custa *O(1)* quando há capacidade e a nova carta fica no fim. Se o valor recebido for menor que todos os anteriores, os *n* elementos são deslocados e o custo chega a *O(n)*. Sob posições de inserção aproximadamente uniformes, o custo médio também é linear. As cópias ocasionadas pelo crescimento geométrico têm custo amortizado constante por inserção, mas uma expansão isolada custa *O(n)* em tempo e memória temporária. Fora da expansão, a inserção usa *O(1)* de espaço de trabalho. Em uma sequência de *n* recebimentos, o custo total das inserções é *O(n+i)*, com *i* inversões na ordem de chegada. A demonstração tem ainda custo quadrático de exibição, pois imprime a mão inteira depois de cada recebimento.

Para o prefixo de entrada `8, 3, 12, 5, 5`, as mãos exibidas são `[8]`, `[3, 8]`, `[3, 8, 12]`, `[3, 5, 8, 12]` e `[3, 5, 5, 8, 12]`. A segunda carta de valor 5 é inserida depois da primeira porque a condição do laço desloca apenas cartas com valor superior.

1.11. Situação 21 - Ordenação cronológica de transações

`MergeSortTransacoes.ordenar` ordena objetos `Transacao` da data e hora mais antiga à mais recente. O algoritmo divide o intervalo ao meio, resolve recursivamente as duas partes e chama `intercalar`. A intercalação copia o intervalo para um vetor auxiliar, lê as duas metades por índices independentes e reescreve o vetor de entrada. O auxiliar é alocado uma vez e reutilizado. `vemAntesOuNoMesmoInstante` seleciona o registro da metade esquerda quando as datas são iguais. Assim, transações registradas no mesmo instante preservam a ordem que tinham na entrada, independentemente do valor financeiro.

As divisões produzem *O(log n)* níveis, e as intercalações de um mesmo nível copiam e escrevem *O(n)* referências ao todo. Como o código executa a intercalação mesmo quando a sequência já está ordenada, o tempo é *O(n log n)* nos casos melhor, médio e pior. O vetor auxiliar ocupa *O(n)*, e a pilha recursiva acrescenta *O(log n)*. O consumo de memória é a contrapartida da estabilidade e do limite de tempo que independe da disposição inicial das transações.

A demonstração cria 50 transações, incluindo `Pagamento 1`, de 120,00, e `Pagamento 2`, de 85,50, nessa ordem e ambas com a data e hora `2026-10-01 14:00:00`. Na listagem, elas aparecem respectivamente nas posições 24 e 25. O resultado mostra que, no empate cronológico, a posição de entrada prevalece sobre o valor da transação.

1.12. Situação 24 - Classificação de atletas por tempo de corrida

`QuickSortAtletas` ordena o vetor do menor para o maior tempo. Quando os tempos coincidem, compara os nomes sem distinção de caixa e, persistindo a equivalência, faz uma comparação sensível à caixa. A rotina recursiva escolhe o elemento central do intervalo como pivô. Dois índices avançam de lados opostos e trocam elementos quando encontram um par a reposicionar. Após o cruzamento, o algoritmo ordena os subintervalos restantes. O vetor permite acessar o pivô e trocar referências em tempo constante. As trocas podem mudar a ordem relativa de registros que continuam equivalentes após todos os critérios.

A validação inicial é linear. Uma partição examina *O(n)* posições no intervalo corrente. Divisões equilibradas levam a *O(n log n)* no melhor caso e, sob uma disposição aleatória de chaves distintas, também no caso médio. O elemento central por posição pode ocupar uma posição extrema na ordem de classificação. Uma sucessão de partições muito desiguais pode levar a $O(n^2)$. Como o código chama recursivamente os dois lados, a memória auxiliar da pilha é *O(log n)* com partições equilibradas e chega a *O(n)* no pior caso. A classificação completa é exibida em *O(n)*, e o pódio lê até três posições do vetor ordenado.

Na demonstração com 30 atletas, o pódio apresenta Paulo em primeiro com 57,84 s, Ursula em segundo com 58,41 s e Davi em terceiro com 58,75 s. Diego e Lucas têm ambos 59,88 s, e Diego é listado antes de Lucas pelo critério de nome. O exemplo distingue a seleção do pódio, que apenas consulta o prefixo, da ordenação de todo o vetor.

2. Implementações em Python

2.1. Situação 02 - Fila de impressão de um laboratório

`FilaDeImpressao` armazena objetos `TrabalhoImpressao` em uma `Fila` encadeada. O número do trabalho é atribuído no envio, e o próximo trabalho a imprimir é retirado da cabeça da fila. O simulador mantém separadamente o trabalho em andamento, o instante de disponibilidade e as listas de trabalhos impressos e cancelados. Ao avançar para um evento, calcula o início como o maior valor entre a disponibilidade da impressora e o horário de envio. A duração é dada pelo número de páginas multiplicado por cinco segundos na configuração da demonstração.

O cancelamento do último trabalho pendente exige uma operação diferente da retirada FIFO. `cancelar_ultimo` transfere todos os trabalhos anteriores para uma fila auxiliar, retira o último e adota a fila auxiliar como principal. Assim, preserva a ordem dos trabalhos restantes e mantém fora do cancelamento o trabalho já em impressão. `enviar`, `proximo` e `imprimir_proximo` custam *O(1)*. Com *q* trabalhos pendentes, cancelar custa *O(q)* em tempo e cria *O(q)* nós na fila auxiliar, reutilizando os objetos dos trabalhos. Em uma fila vazia ou unitária, o cancelamento custa *O(1)*. `listar` percorre os *q* nós e materializa *O(q)* referências. Considerando *e* eventos e *j* trabalhos, o processamento básico é *O(e+j)*, ao qual se somam os tamanhos das filas percorridas nos cancelamentos e nas listagens executadas. Os históricos e a fila somam *O(j)* de armazenamento.

No roteiro de entrada, Ana Beatriz envia um arquivo de 12 páginas às 08h00min05s, e a impressão termina às 08h01min05s. Bruno Henrique, que enviou um arquivo às 08h00min40s, começa a ser atendido no instante em que Ana termina, após 25 segundos de espera. O cancelamento das 08h02min20s retira o trabalho pendente `#05`, de Eduarda Lima. A execução registra 17 envios, dois cancelamentos e 15 impressões, totalizando 161 páginas impressas.

2.2. Situação 04 - Verificação do balanceamento de delimitadores

`verificar_expressao` verifica o aninhamento de parênteses, colchetes e chaves. A classe `Pilha` usa o fim de uma lista nativa como topo. Cada abertura é armazenada com sua posição, iniciada em 1. Ao encontrar um fechamento, a função compara o tipo esperado com a última abertura empilhada. A ordem LIFO corresponde à regra de aninhamento, pois o delimitador aberto mais recentemente é o primeiro que deve ser fechado. O código diferencia fechamento sem abertura, fechamento incompatível e aberturas que sobraram ao final. Neste último caso, desempilha até encontrar a abertura pendente mais antiga.

Sem o registro de passos, a função custa *O(n)* no pior caso, pois percorre os caracteres e empilha ou desempilha cada abertura um número limitado de vezes. As operações no fim da lista têm custo

amortizado constante. O melhor caso para uma entrada não vazia é *O(1)*, como quando o primeiro caractere é um fechamento sem abertura. Se o primeiro erro ocorre após *p* caracteres, o custo é proporcional à parte examinada. Uma expressão válida exige a leitura completa. A pilha usa *O(d)* de memória adicional, em que *d* é a profundidade máxima de aninhamento. Ao receber `passos`, a função também formata e guarda uma representação da pilha a cada operação. Essa opção pode elevar o custo de tempo e armazenamento a $O(n^2)$ em entradas que acumulam aberturas.

A entrada `([)]` produz um resultado inválido na posição 3, com a mensagem `esperava ']' para fechar o '[' da posição 2, mas encontrou ')'`. Com `ignorar_strings=True`, a expressão `print("sorriso :)")` é considerada balanceada. Nesse modo, o algoritmo controla a abertura de aspas simples ou duplas e caracteres escapados, de modo que o parêntese interno ao texto fica fora da verificação. O resultado se refere ao balanceamento dos delimitadores tratados pela função.

2.3. Situação 06 - Controle de estoque com listas encadeadas

`ControleEstoque` mantém uma `ListaEncadeada` de produtos ativos e outra de esgotados. Os códigos são convertidos para maiúsculas e têm os espaços das extremidades removidos. `cadastrar` verifica a unicidade do código nas duas listas e inclui o produto ao fim da lista compatível com o saldo inicial. `atualizar_quantidade` calcula o novo saldo antes de modificá-lo. Se o resultado for negativo, lança `EstoqueInsuficienteError`. Quando o saldo cruza o limite entre zero e um valor positivo, retira o produto de uma lista e insere o mesmo objeto na outra, preservando os dados de cadastro.

As inserções nas extremidades e a religação após um predecessor conhecido custam *O(1)*. No entanto, o cadastro de um código novo verifica as duas listas e custa *O(n)* para *n* produtos existentes. Cadastrar *n* produtos distintos a partir de uma estrutura vazia pode, portanto, custar $O(n^2)$. A busca por código, a remoção e uma movimentação podem se resolver em *O(1)* quando o registro está no primeiro nó examinado, mas custam *O(n)* no pior caso. Uma movimentação entre listas pode procurar o registro uma vez para atualizar o saldo e outra para desvinculá-lo. A busca por nome e `listar_abaixo_do_minimo` percorrem todos os *a* produtos ativos, com *O(a)* visitas e *O(k)* referências de saída para *k* resultados. A busca textual também normaliza os nomes examinados. A estrutura ocupa *O(n)* nós e usa *O(1)* de espaço de trabalho nas alterações, com comprimentos de chaves limitados.

Na simulação, a torneira `TOR-12` começa com 12 unidades. Duas saídas, de quatro e oito unidades, reduzem o saldo a zero e fazem o produto aparecer entre os esgotados. A entrada posterior de 20 unidades reintegra o registro à lista de ativos. O relatório final exibe quantidade 20, mínimo 5 e valor em estoque de R$ 558,00. A divisão em listas torna explícita a situação de cada produto. Um índice por código reduziria o custo das buscas pontuais, com a contrapartida de manter mais uma estrutura coerente com as movimentações.

2.4. Situação 08 - Autocompletar nomes de produtos

`AutocompleteProdutos` insere os nomes em `ArvoreBST`, usando a versão normalizada como chave e o nome original como valor. A normalização converte para minúsculas, decompõe os caracteres e remove as marcas de acentuação. Isso permite consultar `cafe` e obter nomes grafados com `Café`. Se dois nomes resultarem na mesma chave, `ArvoreBST.inserir` atualiza o valor existente e retorna `False`. A busca em `sugerir` explora o fato de os nomes com um prefixo comum ocuparem um intervalo na ordenação lexicográfica. Ao encontrar um nome menor que o prefixo que não seja uma correspondência, segue para a direita. Se o nome é maior e tampouco corresponde, segue para a esquerda. Ao encontrar uma correspondência, explora os lados pertinentes em ordem e respeita o limite solicitado.

Com altura *h* e *k* sugestões devolvidas, a consulta visita *O(h+k)* nós, pois percorre caminhos de fronteira e os nós correspondentes necessários à saída. O custo da normalização do prefixo e das operações de comparação textual soma-se a essa contagem. A recursão usa *O(h)* de memória, e a lista de sugestões ocupa *O(k)*. Inserção, busca pontual e remoção têm limite *O(h)*. As duas primeiras são iterativas, e a remoção usa até *O(h)* de pilha. Uma árvore de altura logarítmica favorece a poda, enquanto a inserção de chaves em ordem crescente pode produzir *h=n*. O método `altura` calcula esse valor por travessia, com *O(n)* de tempo. Uma árvore balanceada evitaria a degeneração. Uma trie, por sua vez, organizaria a busca por caracteres, com outra distribuição de memória entre nós e referências.

Na base inicial com 38 produtos, `sugerir("cafe", limite=5)` visita dez nós e devolve `Café em cápsulas`, `Café torrado e moído 500 g` e `Cafeteira elétrica`, nessa ordem. Após o cadastro de `Cafeteira italiana`, a mesma busca devolve também o novo nome, na quarta posição. A demonstração insere os 38 nomes em ordem alfabética em outra árvore e obtém altura 38. Nessa estrutura, a consulta `smart` visita 34 nós, contra seis na árvore formada pela ordem inicial de cadastro.

2.5. Situação 10 - Ranking de jogadores com AVL e tabela hash

`RankingJogadores` combina uma `ArvoreAVL`, ordenada pela chave `(-pontos, nome)`, com uma `TabelaHash` que associa cada nome à pontuação vigente. O sinal negativo faz com que o percurso crescente comece pelos maiores placares, e o nome desempata pontuações iguais. Quando a pontuação muda, `atualizar` obtém a chave antiga pela tabela, retira o nó correspondente e insere a nova chave na árvore. Essa sequência preserva a ordenação, que poderia ser violada se a chave fosse modificada no próprio nó. `somar_pontos` também aplica o limite inferior de zero ao placar. As rotações mantêm a altura e o tamanho de cada subárvore, permitindo a `posicao` contar quantos jogadores precedem uma chave e a `k_esimo` escolher uma posição por descidas na árvore.

A AVL ocupa *O(n)* nós. Busca por posição, consulta do *k*-ésimo colocado e alterações na árvore custam até *O(log n)*. A posição pode ser resolvida na raiz em *O(1)*, após a identificação da chave. Inserções e remoções usam *O(log n)* de pilha no pior caso, enquanto `posicao` e `k_esimo` percorrem a árvore iterativamente. Para `top(k)`, com *k* limitado ao total de jogadores, o percurso examina *O(log n+k)* nós para produzir o prefixo. Como `primeiros` usa um gerador recursivo com `yield from`, há também o encaminhamento de cada par pelos níveis ancestrais. Contando esse trabalho, um limite de tempo para a implementação é *O((k+1) log (n+1))*. A pilha usa *O(log n)*, e as listas intermediária e final usam *O(k)*.

A tabela hash usa baldes encadeados e acrescenta *O(C+n)* ao armazenamento, em que *C* é sua capacidade. Consultas e atualizações de chaves existentes custam, em média, *O(1)* com carga controlada, dispersão adequada e chaves de tamanho limitado. Se as chaves se concentram em um balde, essas operações podem custar *O(n)*. Em uma inserção que ultrapassa o limite de carga, a tabela procura um primo a partir do dobro da capacidade e redistribui os nós. A redistribuição percorre os baldes antigos e os registros, aloca o vetor novo e recalcula os índices. A procura do primo adiciona o custo dos testes por divisão. Assim, o limite logarítmico da AVL caracteriza sua parte da operação, enquanto a tabela pode acrescentar trabalho em colisões ou expansões.

Ao registrar Bia e Ana com 100 pontos cada, `top(2)` retorna `[(1, 'Ana', 100), (2, 'Bia', 100)]`. Depois de somar cinco pontos a Bia, o resultado passa a `[(1, 'Bia', 105), (2, 'Ana', 100)]`, e `posicao("Ana")` retorna `2`. O primeiro resultado exemplifica o desempate pelo nome. O segundo mostra a mudança de posição após a remoção e a reinserção da chave de Bia.

2.6. Situação 12 - Rede de amizades e graus de separação

`RedeSocial` representa usuários por vértices e amizades por arestas não direcionadas de `Grafo`. Uma tabela hash traduz nomes em índices, e as listas de adjacência armazenam os vizinhos de cada vértice. `adicionar_aresta` evita laços e examina a lista da origem para evitar uma amizade duplicada. Com os vértices já cadastrados e as consultas hash em condições médias, essa verificação custa até $O(d)$ para uma origem de grau d. A remoção de uma amizade percorre as listas dos dois extremos e pode deslocar itens dentro delas. O grafo ocupa $O(C+V+E)$, com C baldes no índice, V usuários e E amizades. Em uma matriz de adjacência, a consulta por um par de índices seria direta, ao custo de reservar $O(V^2)$ posições.

O grau de separação é calculado por busca em largura. `_bfs` usa uma fila FIFO e registra a distância e o pai quando descobre um vértice, o que evita descobertas repetidas. A primeira descoberta do destino determina um caminho com o menor número de amizades. Para origens e destinos distintos, a inicialização dos vetores custa $O(V)$, mesmo quando o destino é encontrado no primeiro nível. No pior caso, a operação custa $O(V+E)$ em tempo e $O(V)$ de espaço adicional, após resolver os nomes. Para origem e destino iguais, `caminho_mais_curto` devolve imediatamente uma lista unitária após consultar os índices. `componentes` executa buscas a partir dos vértices ainda não visitados e percorre o grafo em $O(V+E)$.

`amigos_em_comum` insere os vizinhos da primeira pessoa em uma tabela hash e consulta nela os da segunda. Para graus d_a e d_b e c amigos comuns, a interseção tem custo esperado $O(d_a+d_b)$ sob dispersão adequada, considerado o custo amortizado da redistribuição dos baldes e separada a procura por capacidades primas. A ordenação final com Merge Sort acrescenta $O(c \log c)$, e as listas e a tabela temporária usam $O(d_a+d_b)$ de memória. `sugerir_conexoes` percorre ainda os vizinhos dos amigos, exclui a pessoa consultada e seus amigos diretos, e contabiliza candidatos por meio de outra tabela. Se s é a soma dos graus dos amigos e r é o número de candidatos, o trabalho esperado de enumeração e ordenação é $O(d+s+r \log r)$ sob as mesmas hipóteses. O recorte é feito após ordenar os candidatos por contagem decrescente e nome normalizado. Colisões concentradas aumentam o custo das operações de tabela nesses dois procedimentos.

Na rede da demonstração, a consulta de Ana a Felipe retorna grau 2 e o caminho `Ana -> Carla -> Felipe`. A consulta inicial de Ana a Paulo retorna grau 8. Após incluir a amizade entre Daniel e Paulo, o caminho passa a `Ana -> Daniel -> Paulo`, com grau 2. Para Ana, a lista de sugestões exibida inclui Eduarda com dois amigos em comum e Felipe com um. Os resultados distinguem a distância medida pela busca em largura da contagem de vizinhos compartilhados usada nas sugestões.

2.7. Situação 13 - Cadastro e autenticação de usuários

`SistemaLogin` indexa cada `ContaUsuario` pelo identificador normalizado com `strip().lower()`. O cadastro impede duplicatas e exige senha de pelo menos seis caracteres. Para cada conta, o código gera um sal de 16 bytes e armazena o resultado de PBKDF2-HMAC-SHA256 com 20.000 iterações. `autenticar` localiza a conta, verifica o bloqueio e, quando necessário, deriva o valor para a senha apresentada e compara os resultados com `hmac.compare_digest`. Após três erros consecutivos, a conta é marcada como bloqueada. Um login correto zera o contador de erros, e `desbloquear` redefine tanto o indicador de bloqueio quanto o contador.

A consulta ao índice tem custo médio $O(1)$ em número de contas, para chaves limitadas e bem distribuídas, e pior caso $O(n)$ se todas as contas estiverem em um balde. A normalização e o cálculo do hash do identificador também dependem do seu comprimento. Para uma conta ativa, a autenticação inclui a codificação da senha e a derivação. Com s bytes de senha e I iterações, esse trabalho pode ser expresso como $O(s+I)$, para sal e saída de tamanhos fixos. Consultas de contas ausentes ou bloqueadas terminam antes da derivação. O cadastro bem-sucedido realiza a derivação e pode ainda acionar a

expansão da tabela, com procura de capacidade prima e redistribuição. O armazenamento persistente em memória é *O(C+n)* em baldes e registros, além dos campos textuais. A derivação usa *O(s)* para a senha codificada e memória de tamanho fixo para os valores criptográficos desse exemplo. `listar_usuarios` percorre baldes e chaves e aplica Merge Sort, com *O(C+n log (n+1))* de tempo e *O(n)* de espaço adicional.

Na demonstração, a conta de Ana Souza é cadastrada com o identificador `ana.souza@exemplo.com.br`. A consulta com `ANA.SOUZA@exemplo.com.br` e a senha correta do conjunto de teste retorna sucesso e `bem-vindo(a), Ana Souza`. O espaço final e as maiúsculas são eliminados pela normalização. No caso de Hugo Martins, a terceira senha incorreta produz `senha incorreta - conta BLOQUEADA após 3 erros`. Uma tentativa com a senha correta antes do desbloqueio retorna `conta bloqueada por excesso de tentativas`. O exemplo demonstra como o estado da conta interfere no fluxo antes do cálculo da senha.

2.8. Situação 16 - Ordenação de uma vitrine por preço

`ordenar_vitrine` faz uma cópia superficial da lista de `ProdutoVitrine` e chama `bubble_sort` com o preço como chave. O parâmetro `crescente` determina a direção. O algoritmo compara vizinhos e, após cada passagem, reduz o trecho ainda em exame. Se uma passagem termina sem trocas, a ordenação é encerrada. Como o comparador usa desigualdade estrita, itens de mesmo preço preservam a ordem de entrada tanto na classificação crescente quanto na decrescente.

No melhor caso, com os itens já dispostos no sentido solicitado, há uma passagem e *n-1* comparações. O tempo é *O(n)*. Para permutações aleatórias de chaves distintas, o número esperado de inversões é quadrático, e cada troca adjacente corrige uma inversão. O caso médio e o pior caso são, portanto, $O(n^2)$. `bubble_sort` usa *O(1)* de espaço de trabalho e registra passadas, comparações, trocas e tempo. A cópia feita por `ordenar_vitrine` acrescenta *O(n)* de tempo e memória e preserva a ordem da lista original. A simplicidade da operação é compatível com uma vitrine de 15 itens, enquanto o crescimento quadrático limita seu uso para catálogos maiores.

Na entrada da demonstração, `Caneca personalizada` aparece antes de `Chaveiro de couro` e ambos custam R$ 39,90. Na saída crescente, ocupam a quarta e a quinta posições, respectivamente. Na decrescente, ocupam a décima primeira e a décima segunda, ainda na mesma ordem relativa. Para a ordem crescente, o programa registra 104 comparações, 60 trocas e 13 passadas.

2.9. Situação 18 - Organização de tarefas por prioridade

`organizar_por_prioridade` aplica Selection Sort a uma cópia da lista de tarefas, usando a chave `(-prioridade, codigo)`. Dessa forma, uma prioridade maior aparece primeiro e, quando as prioridades coincidem, o menor código vem antes. O critério de desempate é importante porque a troca entre posições distantes no Selection Sort pode alterar a ordem relativa de chaves equivalentes. A classe `Tarefa` valida prioridades entre 1 e 5. No escalonamento, `_executar_periodo` percorre a agenda e adia tarefas que ultrapassariam o fim do período, permitindo que uma tarefa posterior, mais curta, ocupe o tempo disponível.

O Selection Sort compara cada elemento do sufixo ainda não ordenado com o melhor candidato corrente. Para *n* tarefas, são *n(n-1)/2* comparações nos casos melhor, médio e pior, com até *n-1* trocas. A ordenação isolada usa *O(1)* de memória de trabalho, e a função que preserva a entrada usa *O(n)* para a cópia. Cada passagem de `_executar_periodo` custa tempo linear no tamanho da agenda recebida e armazena referências às tarefas executadas ou pendentes. Se a manhã começa com *n* tarefas e a tarde recebe *m* tarefas entre pendências e novas entradas, as duas ordenações custam $O(n^2+m^2)$. A política de encaixe segue a ordem de exame e a disponibilidade de tempo, sem explorar combinações de tarefas para maximizar a ocupação do período.

Para as 13 tarefas iniciais, o código registra 78 comparações e dez trocas. As tarefas de prioridade 5 e códigos 3, 8 e 11 aparecem nessa ordem na agenda. Seis tarefas são executadas pela manhã, e sete passam à tarde. A tarefa 5, `Ligar para o fornecedor de papel`, é executada das 11h45 às 12h00 e marcada como `encaixe`, pois uma tarefa anterior da agenda excedia o tempo restante.

2.10. Situação 20 - Atualização diária do ranking de vendedores

`atualizar_ranking` usa `insertion_sort` para ordenar a própria lista de `Vendedor` pelo total acumulado em ordem decrescente. O algoritmo mantém um prefixo ordenado e desloca os vendedores que devem aparecer depois do elemento corrente. Como o deslocamento depende de uma desigualdade estrita, um empate preserva a posição relativa da entrada. Na simulação, a ordem do dia anterior é mantida enquanto os totais são atualizados. A tabela hash `posicao_ontem` registra as colocações anteriores para calcular mudanças exibidas no relatório.

Se há *i* inversões na sequência recebida, o algoritmo executa *i* deslocamentos e custa *O(n+i)* em tempo. O melhor caso, com *i=0*, faz *n-1* comparações. No pior caso, a ordem é inversa à desejada e *i=n(n-1)/2*. Uma permutação aleatória de valores distintos também tem $O(n^2)$ de custo médio. O método de ordenação usa *O(1)* de espaço de trabalho. A simulação mantém ainda cópias da sequência, valores de vendas e o índice de posições anteriores, com *O(n)* de memória adicional para as estruturas usadas a cada dia. O experimento com a cópia embaralhada inclui deliberadamente outra ordenação. Seu custo pode ser quadrático mesmo nos dias em que o ranking real sofre poucas mudanças.

Na segunda-feira da demonstração, os totais dos 12 vendedores são atualizados e Giovana permanece em primeiro, com R$ 61.572,57. O Insertion Sort executa 14 comparações e três deslocamentos sobre a lista na ordem do dia anterior. Para os mesmos vendedores e totais, na cópia embaralhada, são 40 comparações e 34 deslocamentos. A comparação isola o efeito da ordem inicial e mostra em que condição a atualização por inserção é vantajosa.

2.11. Situação 22 - Intercalação de listas de contatos sem duplicatas

`mesclar_sem_duplicatas` combina duas listas ordenadas pelo par formado pelo nome normalizado e pelo e-mail sem espaços nas extremidades e em minúsculas. A função aplica a etapa de intercalação do Merge Sort com um índice por lista. A menor chave disponível é encaminhada a `colocar`, com preferência pela lista A em caso de empate. Como chaves iguais chegam consecutivamente, `colocar` compara o contato recebido com o último registro inserido. Uma ocorrência inédita gera um novo objeto `Contato`. Uma duplicata entra na lista de removidos, e seu departamento é acrescentado ao registro mantido quando a cadeia correspondente ainda não está contida no campo de departamentos. Nome, e-mail e telefone são preservados a partir da primeira ocorrência escolhida.

Para entradas ordenadas com *n* e *m* registros, cada índice avança apenas em uma direção e cada contato é processado uma vez. Com campos de comprimento limitado, o custo é *O(n+m)* nos casos melhor, médio e pior. Os índices e as variáveis locais ocupam *O(1)* fora das saídas. A lista consolidada e a lista de duplicatas ocupam juntas *O(n+m)* posições. Em dados de tamanho variável, o cálculo repetido da chave inclui o custo de normalização. Uma sequência extensa de departamentos agregados ao mesmo contato pode aumentar o custo das buscas por trechos e das concatenações. A função de intercalação pressupõe entradas ordenadas. A demonstração verifica essa precondição e ordena com Merge Sort qualquer lista que a viole, acrescentando até *O(n log n+m log m)* ao processamento.

A demonstração combina 20 contatos do Comercial e 20 do Financeiro, devolvendo 34 contatos únicos e seis duplicatas, com 77 comparações contabilizadas. Carlos Eduardo Nunes é consolidado porque nome e e-mail coincidem. Os dois registros de Ana Paula Souza são mantidos porque têm e-mails diferentes. O

exemplo evidencia que a igualdade é definida pela chave composta, o que permite manter homônimos mesmo quando a normalização produz o mesmo nome.

2.12. Situação 23 - Ordenação de catálogo de comércio eletrônico

`ordenar_catalogo` cria uma cópia da lista de produtos e chama `quick_sort` com a chave correspondente ao critério escolhido. A implementação oferece ordenações por menor preço, maior preço, nome, popularidade e avaliação. Os critérios de preço recorrem ao nome normalizado em caso de empate. As ordenações por popularidade e por avaliação usam uma dessas medidas como primeiro critério e a outra como desempate. Todas as chaves terminam no código do produto, que distingue produtos com códigos diferentes quando os critérios anteriores coincidem. `Produto.__post_init__` calcula o nome normalizado no cadastro ou na leitura, permitindo reutilizá-lo nas comparações do catálogo.

Em `quick_sort.py`, `_mediana_de_tres` ordena o primeiro elemento, o central e o último do trecho, selecionando a mediana como pivô. `_particionar` aplica a partição de Hoare, com índices em sentidos opostos. Elementos iguais ao pivô podem ficar dos dois lados. Para trechos de até três elementos, a rotina usa comparações diretas. Nos demais, `_quick_sort` faz a chamada recursiva sobre a menor parte e continua a maior por meio de um laço. Como o tamanho do trecho efetivamente empilhado cai pelo menos pela metade a cada chamada, a pilha tem limite *O(log n)* mesmo quando as partições geram tempo quadrático. A cópia feita por `ordenar_catalogo` eleva a memória adicional da chamada pública a *O(n)*.

Cada partição percorre linearmente o trecho. Divisões equilibradas dão *O(n log n)* no melhor caso, e o custo esperado para disposições aleatórias de chaves distintas também é *O(n log n)*. A mediana entre três amostras melhora a escolha do pivô em entradas já ordenadas, mas ainda admite partições repetidamente desiguais e pior caso $O(n^2)$. O Merge Sort empregado na comparação divide ao meio, cria fatias e intercala as partes, escolhendo a metade esquerda em caso de empate. Tem tempo *O(n log n)* em todos os casos e espaço auxiliar de pico *O(n)*, incluindo as listas de intercalação. O Quick Sort é instável e usa menos memória de trabalho quando aplicado diretamente a uma lista existente. Já o Merge Sort preserva empates e oferece um limite de tempo independente da ordem inicial.

Em uma entrada com `P003` de nome `B`, preço 50, três vendas e nota 4, `P002` de nome `Café`, preço 25, sete vendas e nota 4, e `P001` de nome `Café`, preço 25, duas vendas e nota 5, `Menor preço` devolve os códigos `P001, P002, P003`. Os dois cafés empatam em preço e nome, e o código resolve a ordem. Para `Mais vendidos`, os mesmos registros produzem `P002, P003, P001`. Na demonstração com o CSV de 1.200 produtos, Quick Sort e Merge Sort produzem a mesma sequência em cada um dos cinco critérios. Em uma entrada ordenada de 500 produtos, a variante demonstrativa com pivô no último elemento realiza 124.750 comparações, enquanto a versão com mediana de três registra 5.204. No teste ordenado com 2.000 produtos, a variante de pivô fixo atinge o limite de recursão. Esses resultados mostram o efeito da escolha do pivô nas entradas testadas e o efeito da estratégia de chamada sobre a profundidade da pilha.

3. Conclusão

As implementações evidenciam que o custo de uma solução depende das operações que o problema exige com maior frequência. Filas encadeadas realizam inserção no fim e retirada no início em tempo constante, o que atende às simulações de clínica e impressão. O cancelamento do último trabalho mostra o custo de adaptar essa disciplina a uma retirada pela extremidade oposta. Pilhas associam naturalmente o topo à alteração mais recente de um editor ou à abertura ainda pendente de uma expressão. No editor, a restauração em tempo constante convive com o custo de armazenar versões do texto. Na verificação de expressões, o espaço é determinado principalmente pela profundidade de aninhamento e, quando solicitado, pelo volume do histórico de diagnóstico.

A escolha de um índice depende da relação entre consultas pontuais e operações ordenadas. As tabelas hash são adequadas para frequências, credenciais e localização de registros quando as chaves se distribuem pelos baldes. Árvores binárias de busca permitem emitir resultados ordenados e eliminar subárvores incompatíveis com um prefixo. As árvores AVL acrescentam previsibilidade diante de cadastros em ordem desfavorável. A combinação de uma AVL com um índice hash no ranking de jogadores atende a duas consultas diferentes, a identificação pelo nome e a posição na classificação. Em problemas de grafos, a representação por adjacência atende tanto a relações sociais quanto a rotas de entrega. A busca em largura minimiza o número de arestas, enquanto Dijkstra minimiza a soma de pesos não negativos. O significado atribuído ao custo da conexão é, portanto, parte da escolha do algoritmo.

Nos problemas de ordenação, as comparações e as movimentações têm efeitos distintos. Selection Sort limita trocas e mantém um número quadrático de comparações. Bubble Sort com parada antecipada é apropriado para coleções pequenas ou que podem chegar já ordenadas. A inserção é especialmente adequada quando se mantém um prefixo ordenado ou quando poucas inversões surgem entre atualizações. Merge Sort oferece estabilidade e tempo previsível, propriedades pertinentes à cronologia de transações, e sua etapa de intercalação resolve em tempo linear a consolidação de listas previamente ordenadas. As variantes de Quick Sort mostram que a escolha do pivô e a organização das chamadas recursivas controlam aspectos diferentes do desempenho. A primeira afeta o equilíbrio das partições, e a segunda determina quanta memória a execução pode ocupar na pilha.

Em sistemas reais, essas relações orientam escolhas como manter a ordem de chegada de solicitações, localizar contas, atualizar classificações, consolidar registros ou buscar caminhos. A estrutura mais adequada é aquela cujas invariantes correspondem às regras do domínio e cujos custos permanecem aceitáveis quando variam o volume, a ordem inicial e a frequência das consultas. Avaliar conjuntamente o comportamento funcional, o tempo de processamento e a memória necessária permite justificar essas escolhas com base nas operações que o software efetivamente executa.